Repaso de Funciones Totales y `Maybe`

Paradigmas (de Lenguajes) de Programación

26 de Agosto de 2025

1 Funciones Totales

Las funciones transforman un valor de entrada a un valor de salida. Cuando caracterizamos los valores como conjuntos, por ejemplo usando tipos de datos, podemos hablar de funciones totales y parciales. Llamamos **totales** a las funciones cuyo dominio coincide con el conjunto de partida, y **parciales** a las funciones cuyo dominio es un subconjunto propio de dicho conjunto.

Las funciones totales siempre devuelven un valor. Esto es **muy conveniente** a la hora de analizar código porque no tenemos que preocuparnos si los argumentos son correctos.

Muchas veces analizamos los programas con la mentalidad de que "si termina entonces ...". Ese "si termina" contempla la precondición. Si no satisfacemos la precondición, entonces perdemos las garantías de cómo se puede comportar el programa.

La función suma **es total**:

```
(+) :: Num a => a -> a -> a
```

Si `x, y :: Int` sabemos que `x + y :: Int` sin importar el valor de `x` e `y` y sabemos que el programa `x + y` va a terminar.

La función de división entera **es parcial**

```
div :: Integral a => a -> a -> a
```

Su precondición es que el segundo argumento sea distinto de 0. Si `x, y :: Int` entonces

`div x y` no tiene comportamiento definido si `y == 0`, en particular da error deteniendo la ejecución de todo el programa.

Otra función que ya conocen es `head :: [a] -> a`. ¿Será total o parcial? ¿Tiene alguna precondición no trivial? ¡Es Parcial! la lista tiene que no ser vacía.

Sigamos analizando `(+)` y `div` como ejemplos más sencillos. Pero las herramientas sirven también para otras estructuras de datos.

Supongamos que queremos hacer un programa que calcula $x/y + z$.

```
programa :: Int -> Int -> Int -> Int
programa x y z = div x y + z
```

En principio, si no sabemos los detalles de la implementación, viendo solo el tipo podríamos pensar que cualquier valor es válido. Pero la precondición (que no siempre está explícita) nos dice que $y \neq 0$. De hecho, como programadores del `programa` podríamos no darnos cuenta porque la división bien podría estar siendo llamada dentro de otras funciones en lugar de estar directamente a la vista.

En definitiva, mientras más información nos provean los tipos de datos, mejor. Porque el chequeo de tipos no sabe de precondiciones, solo de tipos.

Importante

Trabajar con funciones totales empuja más información al sistema de tipos.

2 ¿Cómo obtener una función total?

Dada una función parcial, a veces puede obtenerse una función total que se comporte de la misma forma para los valores definidos. Esto se logra modificando el dominio o la imagen de la función.

Veamos cómo sería un ejemplo de `safeDiv` para `Int`.

2.1 Modificando la imagen

Si modificamos la imagen de la función, su resultado, podemos hacer que en lugar de calcular un `Int` calculemos si la operación tuvo éxito y en tal caso tener un int. El tipo `Maybe` puede usarse para tal fin.

```haskell
data Maybe a = Just a | Nothing

safeDiv :: Int -> Int -> Maybe Int
safeDiv = _
```

No nos importa la implementación, sino analizar la consecuencia del cambio en la función: `safeDiv` puede ser total! `safeDiv x y` va a ser un `Maybe Int` y terminar sin importar los valores de los argumentos. Cuando `y == 0` obtendremos un `Nothing` y en otro caso un `Just` con el resultado de la división.

Ahora bien, si tratamos de usar `safeDiv` en nuestro programa vamos a tener un problema.

```haskell
programa :: Int -> Int -> Int -> Int
programa x y z = safeDiv x y + z
```

No compila. Porque `(+)` no puede sumar un `Maybe Int` con un `Int`.

Termina siendo tarea de quien usa `safeDiv` decidir qué hacer ante la condición de error. ¡Pero al menos es un error que no detiene todo el programa!

2.1.1 Resolviendo el problema

En principio podríamos resolver la situación dando algún resultado que tenga sentido. Suponiendo que `0` es la respuesta que queremos dar en caso de error, podríamos escribir un `case`.

```haskell
programa :: Int -> Int -> Int -> Int
programa x y z =
  case safeDiv x y of
    Just r -> r + z
    Nothing -> 0
```

2.1.2 Avisando del problema

O bien podríamos querer indicar que hubo un error al evaluar `programa` (usando `Maybe`).

```haskell
programa :: Int -> Int -> Int -> Maybe Int
programa x y z =
  case safeDiv x y of
    Just r -> Just r + z
    Nothing -> Nothing
```

Tip

Conocer o armar algunas funciones sobre `Maybe` es útil para lidiar con estas situaciones en forma más cómoda.

2.2 Modificando el dominio

Otra alternativa es tener un tipo de dato que represente a los enteros sin el cero, por ejemplo `NonZero`. De esta forma podemos modificar el dominio, los argumentos de `safeDiv` para que quede de la siguiente forma.

```
safeDiv :: Int -> NonZero -> Int
safeDiv = _
```

El tipo de dato `NonZero` estaría declarado en otro módulo y tendrá alguna forma de obtener un valor `NonZero`. La siguiente podría ser su interfaz.

```
module NonZero where (NonZero, maybeNonZero, toInt)

data NonZero

maybeNonZero :: Int -> Maybe NonZero

toInt :: NonZero -> Int
```

Nuevamente, si reemplazamos `div` por `safeDiv`, el programa va a tener un problema de tipos, ya que `y` debería ser `NonZero` en esta versión de `safeDiv`. Como usuarios de `safeDiv` nos vemos obligados a realizar la verificación sobre `y`. Usando `maybeNonZero` podemos hacer dicha verificación y decidir qué resultado deberíamos dar en tal caso.

```
programa :: Int -> Int -> Int -> Int
programa x y z =
    case maybeNonZero y of
      Just y' -> safeDiv x y' + z
      Nothing -> 0
```

O, si preferimos, indicar el error a los usuarios de `programa`.

```
programa :: Int -> Int -> Int -> Maybe Int
programa x y z =
    case maybeNonZero y of
      Just y' -> Just (safeDiv x y' + z)
      Nothing -> Nothing
```

Nuevamente estamos en control de cómo resolver la situación que antes era un error que frenaría todo el programa.

3 Maybe

Tip En otros lenguajes...

A `Maybe` se lo suele comparar con `null`, `nil`, `None` de lenguajes como Java, Ruby o Python. Pero hay una diferencia importante. En muchos lenguajes es usual que una variable de tipo `String` pueda tener un valor no inicializado `null`. En estos lenguajes entonces `null` es un posible valor del tipo `String`. Expresiones como `append("hola", null)` pasan el chequeo de tipos, pero pueden no evaluar satisfactoriamente.

En Haskell, lo equivalente, `"hola" ++ Nothing` no tipa. `"hola" :: String` y `Nothing :: Maybe a`, la función `(++)` no está definida. Una función que opere con `String` no puede operar con `Maybe String`, hay que adaptarla.

Esto hace que a veces tengamos:

- Que aplicar `Just` a un string para pasarlo por parámetro,
- O que necesitemos "desarmar" un `Maybe` para saber qué tiene adentro,
- O que usemos alguna función que nos resuelva la fricción que puede introducir el `Maybe`

El tipo `Maybe` puede ser útil en muchos contextos y conocer algunos patrones usuales puede ayudar.

3.1 Valor por defecto

Cuando tenemos un `Maybe a` y queremos sí o sí un `a` hay dos opciones:

- O bien usamos una función **parcial** `fromJust :: Maybe a -> a` que se va a indefinir si el argumento es `Nothing`,
- o bien usamos una función **total** `fromMaybe :: a -> Maybe a -> a` que devuelve el primer argumento si el segundo es `Nothing`.

Ambas funciones están ya definidas en `Data.Maybe` del paquete `base`, pero su definición

es muy sencilla:

```haskell
fromJust :: Maybe a -> a
fromJust (Just v) = v
-- Función parcial, no hay "fromJust Nothing = ..."

fromMaybe :: a -> Maybe a -> a
fromMaybe _ (Just v) = v
fromMaybe d Nothing = d
-- Función total!
```

3.2 Transformar

Hay veces que necesitamos transformar el valor dentro de un `Maybe` y mantener el `Nothing`. En las versiones de `programa :: Int -> Int -> Int -> Maybe Int`, esto mismo fue lo que terminó pasando.

Podemos definir `mapMaybe`, que aplique la transformación de `a` a `b`.

```haskell
mapMaybe :: (a -> b) -> Maybe a -> Maybe b
mapMaybe _ Nothing = Nothing
mapMaybe f (Just a) = Just (f a)
```

Luego reescribimos el `programa` con `safeDiv :: Int -> Int -> Maybe Int` evitando el case.

```haskell
programa :: Int -> Int -> Int -> Maybe Int
programa x y z =
  (safeDiv x y) `mapMaybe` (\r -> r + z)
```

Tip

Lo cómodo de las funciones de alto orden como `mapMaybe` es que aplicar varias de estas suele ser más cómodo que anidar `case`.

La función `mapMaybe` ya está definida como `fmap` en `Prelude`. Si bien tiene un tipo más general `fmap :: Functor f => (a -> b) -> f a -> f b` puede usarse con `Maybe`.

3.3 Promover

Supongamos queremos hacer un programa que use dos divisiones $a/b + x/y$. Si alguna de las divisiones falla, el programa entero debe fallar.

Hay varias formas de hacerlo. Un intento directo de usar `safeDiv` por como división nos daría un problema de tipos nuevamente.

```
programa :: Int -> Int -> Int -> Int -> Maybe Int
programa a b x y = (safeDiv a b) + (safeDiv x y)
--                                error ---^
```

Podemos pensar en promover la suma `(+)` de `Int` a `Maybe Int` y si algunos de los argumentos es `Nothing` entonces el resultado es `Nothing`. Esta idea puede aplicar a cualquier operación de dos argumentos, incluso para distintos tipos de argumentos.

Definiendo `lift2` de la siguiente manera:

```
lift2 :: (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c
lift2 _ Nothing _ = Nothing
lift2 _ _ Nothing = Nothing
lift2 f (Just a) (Just b) = Just (f a b)
```

Podemos escribir:

```
programa :: Int -> Int -> Int -> Int -> Maybe Int
programa a b x y = lift2 (+) (safeDiv a b) (safeDiv x y)
```

La función `lift2` ya está definida como `liftA2` en `Prelude`. Si bien tiene un tipo más general `liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c` puede usarse con `Maybe`.

3.4 Encadenar

Otra opción que sirve en el caso anterior es pensar el programa como una secuencia de cómputos más pequeños y tratar de encadenar dichos cómputos. Si alguno falla, entonces el conjunto también lo hace. De esta forma un cómputo toma un valor como entrada y puede o no tener éxito.

Un cómputo entonces sería de la forma `a -> Maybe b`.

Dado el **resultado** de un primer cómputo (de tipo `Maybe a`), vamos a querer encadenar un segundo cómputo (de tipo `a -> Maybe b`) y obtener su el resultado, que puede fallar.

Notar que estamos suponiendo que el segundo cómputo solo va a ser ejecutado si el primero tiene éxito, ya que toma un `a` en lugar de un `Maybe a`.

A la función que necesitamos podemos llamarla `andThen`.

```
andThen :: Maybe a -> (a -> Maybe b) -> Maybe b
andThen Nothing _ = Nothing
andThen (Just a) f = f a
```

Tip

Se parece a `mapMaybe` pero no es igual.

Al usarla podemos ver que hay tres cómputos: a/b, x/y y la suma de ambos. Cada `andThen` nos permite darle un nombre al resultado del cómputo anterior y dicho valor va ser en este caso un `Int`, no un `Maybe Int`, quitando la necesidad de promover la suma.

```
programa :: Int -> Int -> Int -> Int -> Maybe Int
programa a b x y =
    (safeDiv a b) `andThen` (\ aDivB ->
        (safeDiv x y) `andThen` (\ xDivY ->
          Just (aDivB + xDivY)
        )
    )
```

Notar que por como se encadenan los cómputos, el último tiene acceso al resultado de todos los cómputos anteriores.

La función `andThen` ya está definida como `(>>=)` en `Prelude`, se le dice *bind*. Si bien tiene un tipo más general `(>>=) :: Monad m => m a -> (a -> m b) -> m b`, puede usarse con `Maybe`.

Tip Notación `do` (extra)

Alineando el código un poco distinto puede dar la sensación de "asignación". De hecho al usar `(>>=)` se puede usar también la función `pure`, que en este contexto tendra tipo `a -> Maybe a` y equivale a `Just`.

```
programa :: Int -> Int -> Int -> Int -> Maybe Int
programa a b x y =
    (safeDiv a b) >>= (\ aDivB ->
    (safeDiv x y) >>= (\ xDivY ->
        pure (aDivB + xDivY)
    ))

```

En Haskell la clase `Monad`, con las funciones `(>>=)` y `pure`, permite usar la notación `do`. El siguiente código es equivalente 100% al anterior pero se lee más fácil.

```
programa :: Int -> Int -> Int -> Int -> Maybe Int
programa a b x y = do
    aDivB <- safeDiv a b
    xDivY <- safeDiv x y
    pure (aDivB + xDivY)

```

3.5 Alternativa

Otro patrón útil se usa cuando tenemos dos `Maybe a` y queremos el primero que no sea un `Nothing`.

```
orMaybe :: Maybe a -> Maybe a -> Maybe a
orMaybe (Just a) _ = (Just a)
orMaybe Nothing mb = mb
```

Notemos que es muy similar al *or* de `Bool`, `(||)`. Por eso el nombre `orMaybe`.

```
(||) :: Bool -> Bool -> Bool
(||) True _ = True
(||) False z = z
```

Si tenemos que determinar el primero de tres valores que no sea `Nothing`, podemos simplemente escribir lo siguiente sin necesidad de `case` y otras funciones auxiliares.

```
( a `orMaybe` b ) `orMaybe` c
```

3.6 Otros

Hay más patrones útiles que pueden aparecer dependiendo de cómo pensemos los problemas. Definir funciones auxiliares para abstraer el comportamiento deseado en cada situación es un recurso válido para lograr un programa más fácil de entender.

4 Chequeando precondiciones

Volvamos al programa del principio: $x/y + z$

```
programa :: Int -> Int -> Int -> Int
programa x y z = div x y + z
```

Podríamos hacerlo total también si tenemos cuidado con cuándo usamos `div`.

```
programa :: Int -> Int -> Int -> Maybe Int
programa x y z =
    if y /= 0
        then Just (div x y + z)
        else Nothing
```

Pero **no estamos aprovechando al información de tipos** para asegurarnos de que `programa` sea total. Al usar una función parcial debemos analizar el programa cuidadosamente. Este programa es corto, pero se puede volver un análisis complicado muy rápido.

Distinto es cuando usamos funciones totales. **La composición de funciones totales siempre es total**. En el siguiente programa, al usar todas funciones totales, estamos seguros de que el programa va a terminar para toda entrada. No hay ningún análisis necesario.

```
programa :: Int -> Int -> Int -> Maybe Int
programa x y z =
    (safeDiv x y) `andThen` (\ xDivY ->
        Just (xDivY + z)
    )
```

Importante

La composición de funciones totales da una función total. La composición de funciones parciales *puede* dar como resultado una función total, pero requiere un análisis manual. Distintos sistemas de tipos corren el límite, pero con los tipos vistos hasta ahora el análisis queda a cargo de la persona.

5 Ejemplo: Recorriendo árboles

Supongamos que dado un árbol binario de enteros (sin orden) queremos saber si algún `Int` que cumpla cierta condición está o no definido (contenido) en dicho árbol.

La condición pues está representada como `Int -> Bool`.

Si encontramos un elemento, queremos obtener cuál fue dicho `Int`. El resultado pues de la función podría ser `Maybe Int` para indicar si lo encontramos o no y en tal caso cuál es.

```
data ABInt = Bin ABInt Int ABInt | Nil

definido :: ABInt -> (Int -> Bool) -> Maybe Int
definido ab f = _
```

Una versión directa de tal función podría ser:

```
definido :: ABInt -> (Int -> Bool) -> Maybe Int
definido Nil f = Nothing
definido (Bin i n d) f =
    if f n
    then Just n -- es la raíz
    else
        case definido i f of
            Just ri -> Just ri -- está en i
            Nothing ->
                case definido d f of
                    Just rd -> Just rd -- está en d
                    Nothing -> Nothing -- no está
```

Mucho anidamiento. El último `case` no hace falta en rigor, igual: evalúa al valor de `definido d f`.

```haskell
definido :: ABInt -> (Int -> Bool) -> Maybe Int
definido Nil f = Nothing
definido (Bin i n d) f =
    if f n
    then Just n -- es la raíz
    else
        case definido i f of
            Just ri -> Just ri -- está en i
            Nothing -> definido d f -- está o no en d
```

Igual entender que realiza una búsqueda *preorder* (primero la raíz, luego en rama izquierda y luego en rama derecha) no es trivial de ver.

Reescribámoslo usando algunas de las operaciones de `Maybe` vistas antes.

```haskell
definido :: ABInt -> (Int -> Bool) -> Maybe Int
definido Nil f = Nothing
definido (Bin i n d) f =
    (if f n then Just n else Nothing) `orMaybe`
    ((definido i f) `orMaybe` (definido d f))
```

Sabiendo que `orMaybe` evalúa al primer argumento si este no es `Nothing` podemos leer (y cambiar) el orden en que se recorre el árbol con mayor facilidad.